

Guia de Estudo PAW — Ordem Estratégica para 20/20

Baseado na análise de 3 exames anteriores (2020/21, 2021/22, 2024/25) e nos 13 PDFs da disciplina. Cada secção indica: matéria, o que sai no exame, e exercícios reais de anos anteriores.

Análise dos Exames — O que REALMENTE cai?

Antes de estudar, é importante saber onde estão os pontos:

Parte do Exame	Peso	O que testa
Escolha Múltipla	7-8 valores	Conceitos teóricos, armadilhas, identificar código
Verdadeiro/Falso	2-3 valores	Afirmações traiçoeiras com justificação
Resposta Aberta	10 valores	Explicar conceitos, corrigir código, descrever arquiteturas

Temas que aparecem TODOS os anos (garantido):

Tema	Nº de vezes em 3 exames	Tipo de pergunta
NPM — o que faz vs não faz	3/3	Escolha múltipla
Express — server-side, NÃO corre no browser	3/3	Escolha múltipla
Angular — client-side, NÃO é fullstack	3/3	Escolha múltipla / V/F
Serviços Angular (Injectable, Observable)	3/3	Escolha múltipla + Aberta
JWT — autenticação + autorização	3/3	Escolha múltipla + Aberta
HTML/CSS NÃO são linguagens de programação	3/3	V/F
Validação frontend + backend	3/3	V/F + Aberta
Modelo cliente-servidor	2/3	Aberta (1-1.5 val)
Corrigir erros em HTML/JS	2/3	Aberta (1-2 val)
MVC — explicar o padrão	2/3	Aberta (1.5 val)
REST API — explicar + CRUD	2/3	Escolha múltipla + Aberta
Guards vs Interceptors	2/3	Aberta (1.5 val)
localStorage para persistência	2/3	Aberta (1 val)
Identificar tecnologia por código	3/3	Escolha múltipla

FASE 1 — Fundamentos (≈2h de estudo)

🎯 **Objetivo:** Construir as bases. Sem isto, o resto não faz sentido. 📄 **PDFs:** Introdução ao HTML.pdf, Introdução ao CSS.pdf

1.1 — Conceitos Fundamentais da Web

Modelo Cliente-Servidor ★★

Saiu em: Aberta 2024/25 (1 val), Aberta 2020/21 (1.5 val)

A arquitetura cliente-servidor é o modelo base de toda a web:

CLIENTE (Browser):

- O browser do utilizador (Chrome, Firefox, etc.)
- Executa código HTML, CSS, JavaScript (e frameworks como Angular)
- Inicia a comunicação enviando pedidos HTTP
- Responsável pela interface com o utilizador (frontend)
- Não deve ter acesso direto a bases de dados nem a credenciais privadas

SERVIDOR:

- Aplicação que corre num computador remoto (ex: Node.js + Express)
- Recebe pedidos HTTP, processa-os e devolve respostas
- Accede a bases de dados, APIs externas e recursos protegidos
- Responsável pela lógica de negócio, segurança e dados (backend)
- Pode servir páginas HTML (SSR) ou dados JSON (API REST)

Fluxo típico (saber de cor para a resposta aberta):

1. Utilizador escreve URL ou clica num link → browser envia pedido HTTP ao servidor
2. Servidor processa o pedido (middleware → controller → model/BD)
3. Servidor devolve resposta HTTP (HTML, JSON, código de status)
4. Browser renderiza a resposta ao utilizador

Protocolo HTTP ★★

Método	Propósito	Corpo?	Idempotente?	Operação CRUD
GET	Ler dados	Não	Sim	Read
POST	Criar dados	Sim	Não	Create
PUT	Atualizar dados	Sim	Sim	Update
DELETE	Eliminar dados	Opcional	Sim	Delete

⚠ Armadilha de exame (2020/21 P2): CREATE e UPDATE NÃO são métodos HTTP! São operações CRUD. Os métodos HTTP equivalentes são POST e PUT.

Códigos de Status HTTP:

Gama	Significado	Exemplos
2xx	Sucesso ☑	200 OK, 201 Created, 204 No Content
3xx	Redirecionamento	301 Moved, 302 Found
4xx	Erro do cliente ✗	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
5xx	Erro do servidor ✨	500 Internal Server Error

⚠ Armadilha: 401 Unauthorized = não autenticado (falta login). 403 Forbidden = autenticado mas sem permissão (falta autorização). São diferentes!

Distinções Fundamentais ★★

Saiu em: V/F em TODOS os exames

Conceito	NÃO é	É
HTML	Linguagem de programação	Linguagem de marcação (estrutura)
CSS	Linguagem de programação	Linguagem de estilos (apresentação)
JavaScript	Java	Linguagem de programação (lógica)

Conceito	NÃO é	É
Node.js	Linguagem / Framework / Servidor web	Runtime para JavaScript
Express	Linguagem	Framework web para Node.js
Angular	Framework fullstack	Framework client-side (frontend)
NPM	Compilador / Runtime	Gestor de packages para JS
MongoDB	Base de dados relacional	Base de dados NoSQL (documentos)
Mongoose	Base de dados	Driver/ODM para MongoDB em Node.js

 **DECORAR:** Estas distinções aparecem em **TODOS** os exames como armadilhas de escolha múltipla e V/F.

1.2 — HTML

Estrutura básica

```
<!DOCTYPE html>          <!-- Tipo de documento (HTML5) -->
<html lang="pt">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Título</title>
  </head>
  <body>
    <h1>Título Principal</h1>
    <p>Parágrafo</p>
  </body>
</html>
```

Conceitos-chave

Conceito	Descrição
Block-level	Inicia nova linha, ocupa toda a largura (<div>, <p>, <h1>, , <table>)
Inline	Não inicia nova linha, ocupa só o necessário (, <a>, ,)
Atributos	Declarados em minúsculas, valores entre aspas (src, alt, href, id, class)

Tags importantes

- Cabeçalhos: <h1> a <h6>
- Links: Texto
- Imagens:
- Listas: (não ordenada) / (ordenada) +
- Tabelas: <table>, <tr>, <th>, <td>, colspan, rowspan
- Formulários: <form action="/rota" method="POST">, <input>, <button>
- type: text, password, radio, checkbox, submit

Formulários e GET vs POST ★★

Saiu em: V/F 2021/22, Aberta 2021/22, Escolha múltipla 2020/21

- NUNCA usar GET para dados sensíveis (como login/passwords), pois o GET expõe os dados no URL
- Um formulário HTML pode submeter dados sem JavaScript — basta usar <form action="/rota" method="POST">
- O conteúdo do <head> NÃO é visível na página (metadados)

HTML Semântico (HTML5)

<header>, <nav>, <main>, <section>, <article>, <aside>, <footer>, <figure>, <figcaption>

Elementos multimédia: <video>, <audio>, <canvas>, <svg>

1.3 — CSS

3 formas de adicionar CSS

Saiu em: Aberta 2020/21 (1.5 val)

- Inline: <tag style="color: red;">
- Internal: <style> dentro do <head>
- External: <link rel="stylesheet" href="ficheiro.css"> ★ (recomendado)

Seletores

Seletor	Sintaxe	Aplica-se a
Tag	p { }	Todas as tags <p>
Classe	.nome { }	Elementos com class="nome" (múltiplos)
ID	#nome { }	Elemento com id="nome" (único)

Box Model ★

Conteúdo em camadas — de dentro para fora:

Content → Padding → Border → Margin

Media Queries (Web Responsiva)

```
@media (max-width: 480px) { /* phones */ }
@media (max-width: 720px) { /* tablets */ }
@media (min-width: 992px) { /* laptops */ }
```

FASE 2 — JavaScript & DOM (≈2.5h de estudo)

🎯 *Objetivo: Dominar JS e manipulação DOM — as perguntas de "corrigir código" vêm daqui.* 📄 *PDFs: Introdução ao Javascript.pdf, Javascript e HTML DOM.pdf*

2.1 — JavaScript Essencial

Variáveis: var vs let vs const ★★ ★

Saiu em: Armadilha frequente em escolha múltipla

```
var nome = "João";           // scope de FUNÇÃO + hoisting
let idade = 20;               // scope de BLOCO (ES6)
const PI = 3.14;              // constante, não pode ser reatribuída (ES6)
```

⚠ *var sofre hoisting (a declaração "sobe" ao topo da função). let e const não.*

Operadores de Comparação ★ ★

Operador	Significado
==	Igual (valor, com coerção de tipo)
===	Igual (valor E tipo) ★
!=	Diferente
!==	Diferente (valor OU tipo)

Usar sempre === em vez de == — boa prática obrigatória.

Funções

```
// Function Declaration (sofre hoisting)
```

```
function soma(a, b) { return a + b; }

// Function Expression
const soma = function(a, b) { return a + b; };

// Arrow Function (ES6)
const soma = (a, b) => a + b;
```

Promises e Async/Await ★

```
// Promise
const promise = new Promise((resolve, reject) => {
  if (sucesso) resolve(resultado);
  else reject(erro);
});
promise.then(res => {}).catch(err => {});

// Async/Await (mais legível)
async function obterDados() {
  try {
    const res = await fetch('/api/dados');
    const dados = await res.json();
  } catch (err) {
    console.error(err);
  }
}
```

Estados de uma Promise: pending → fulfilled ou rejected

2.2 — DOM, AJAX, JSON & Storage

Aceder a Elementos DOM ★★

Saiu em: Escolha múltipla 2024/25, Aberta 2024/25, Aberta 2020/21

Método	Retorna
document.getElementById(id)	UM elemento (o primeiro)
document.getElementsByTagName(tag)	Coleção
document.getElementsByClassName(classe)	Coleção
document.querySelector(seletor)	Primeiro match CSS
document.querySelectorAll(seletor)	Todos os matches CSS

🕸 **Armadilha clássica:** `getElementById` retorna UM ÚNICO elemento (o primeiro), NÃO todos! IDs devem ser únicos no HTML.

Manipular Elementos

```
element.innerHTML = "Novo conteúdo"; // Altera conteúdo HTML
element.textContent = "Texto puro"; // Altera texto (sem HTML)
element.setAttribute("class", "destaque"); // Altera atributo
element.style.color = "red"; // Altera estilo CSS
```

⚠ `innerHTML` interpreta HTML; `textContent` trata como texto puro.

AJAX (XMLHttpRequest) ★★

Saiu em: V/F 2021/22

```
const xhttp = new XMLHttpRequest();
xhttp.onreadystatechange = function() {
  if (this.readyState == 4 && this.status == 200) {
    // resposta em this.responseText
  }
}
```

```
};
xhttp.open("GET", "url", true); // true = assíncrono
xhttp.send();
```

AJAX permite atualizar partes da página sem recarregar toda a página. `readyState == 4` = resposta completa.

localStorage vs sessionStorage ★ ★ ★

Saiu em: Aberta 2024/25 (P3.2) — "como manter dados entre recarregamentos"

Característica	localStorage	sessionStorage
Persistência	Permanente (sobrevive ao fechar browser)	Apenas durante a sessão/tab
Scope	Mesmo domínio, qualquer tab	Apenas a tab atual

```
localStorage.setItem("chave", "valor");
localStorage.getItem("chave");
localStorage.removeItem("chave");
```

Padrão típico de exame:

```
// 1. Ao carregar a página, ler dados guardados
var dados = JSON.parse(localStorage.getItem('meusDados')) || valorPadrao;
// 2. Mostrar dados no HTML
document.getElementById('elemento').textContent = dados;
// 3. Ao alterar dados, guardar no localStorage
function atualizar(novoDado) {
  dados = novoDado;
  document.getElementById('elemento').textContent = dados;
  localStorage.setItem('meusDados', JSON.stringify(dados));
}
```

Exercício Real — Corrigir Código HTML/JS

Saiu em: Aberta 2024/25 (P3.1, 1 valor), Aberta 2020/21 (P4, 2 valores)

```
<html>
<body>
  <h1>Click Counter</h1>
  <p>Click Number: <span id="contador">0</span></p>
  <button>Click Me</button>
  <script>
    function contarClique() {
      numeroClique++;
      document.getElementById(counter).textContent = numeroClique;
    }
  </script>
</body>
</html>
```

3 erros clássicos (memorizá-los!):

- ✗ `numeroClique` nunca é declarado → `var numeroClique = 0;`
- ✗ `getElementById(counter)` — `counter` é variável undefined → Deve ser string: `getElementById("contador")`
- ✗ O botão não tem evento associado → Adicionar `onclick="contarClique()"` ou usar `addEventListener`

FASE 3 — Backend: Node.js, Express, BD & REST (≈3h de estudo)

🎯 **Objetivo:** Dominar a stack backend — a parte com MAIS peso nas respostas abertas. 📄 **PDFs:** Introdução ao Backend em NodeJS.pdf, Express Framework.pdf, NodeJS Express DataStorage.pdf, NodeJS Express REST API.pdf

3.1 — Node.js

O que é Node.js? ★★

- Runtime JavaScript para executar JS fora do browser (servidor/desktop)
- Não é um servidor web por si só — é preciso codificar um servidor HTTP
- Single-threaded com I/O assíncrono (não bloqueante)
- Baseado no motor V8 (Chrome)

NPM e package.json ★★

Saiu em: Escolha múltipla em TODOS os exames

```
npm init # Inicializa projeto (cria package.json)
npm install pacote --save # Instala dependência e guarda no package.json
npm start # Executa o script "start" do package.json
```

O que NPM FAZ:

- ✅ Instala packages/módulos
- ✅ Gere dependências
- ✅ Executa scripts (npm start, npm test)

O que NPM NÃO FAZ:

- ❌ NÃO cria componentes Angular (isso é ng generate)
- ❌ NÃO instala MongoDB (instala o driver mongoose)
- ❌ NÃO compila TypeScript (isso é tsc)

3.2 — Express Framework & Middleware

Routing ★★

```
app.get('/rota', (req, res) => { }); // GET
app.post('/rota', (req, res) => { }); // POST
app.put('/rota/:id', (req, res) => { }); // PUT
app.delete('/rota/:id', (req, res) => { }); // DELETE
```

Request: req.params vs req.query vs req.body ★★

Saiu em: Armadilha frequente

Propriedade	Fonte	Exemplo
req.params	Parâmetros de rota (:id)	GET /produto/123 → req.params.id = "123"
req.query	Query strings (?key=val)	GET /search?q=teste → req.query.q = "teste"
req.body	Corpo do pedido (POST/PUT)	Dados enviados via formulário ou JSON

Middleware ★★

Saiu em: Escolha múltipla 2024/25, 2020/21

Funções executadas entre o pedido e a resposta, com acesso a req, res e next().

```
// Middleware ao nível da aplicação
app.use(function(req, res, next) {
  console.log('Pedido recebido');
  next(); // IMPORTANTE: chamar next()!
});

// Middleware de erros (4 parâmetros)
app.use(function(err, req, res, next) {
  res.status(500).send('Erro!');
});
```

```
});
```

Tipos de middleware:

- Aplicação — `app.use()`
- Routing — `router.use()`
- Erros — 4 parâmetros (`err`, `req`, `res`, `next`)
- Built-in — `express.static`, `express.json`, `express.urlencoded`
- Third-party — `body-parser`, `multer`, `cors`

⚠ A ordem do middleware importa! E nunca esquecer de chamar `next()`.

Padrão MVC ★★

Saiu em: Aberta 2024/25 (1.5 val), Aberta 2021/22 (1.5 val), V/F 2020/21

Camada	Pasta	Exemplo
Model	<code>models/</code>	<code>const Produto = mongoose.model('Produto', schema)</code>
View	<code>views/</code>	Ficheiros <code>.ejs</code>
Controller	<code>controllers/</code>	Funções que fazem <code>Produto.find()</code> e <code>res.render()</code>

3.3 — Bases de Dados

SQL vs NoSQL ★★

Aspeto	SQL (Relacional)	NoSQL
Schema	Rígido, bem definido	Flexível, sem schema
Transações	ACID	Consistência eventual
Escalabilidade	Vertical	Horizontal
Exemplos	MySQL, PostgreSQL	MongoDB, CouchDB

CRUD ★★

Saiu em: Aberta 2021/22 (1 val)

- Create → POST → `new Produto().save()`
- Read → GET → `Produto.find()`
- Update → PUT → `Produto.findByIdAndUpdate()`
- Delete → DELETE → `Produto.findByIdAndDelete()`

Autenticação vs Autorização ★★

Saiu em: TODOS os exames

	Autenticação	Autorização
Pergunta	"Quem és tu?"	"O que podes fazer?"
Quando	No login	Após login, em cada pedido
Código HTTP	401 Unauthorized	403 Forbidden
Módulo	<code>jsonwebtoken</code> , <code>passport</code>	<code>Guards</code> , <code>ACL</code> , <code>roles</code>

3.4 — REST APIs & JWT

Propriedades REST ★★

Saiu em: Aberta 2024/25 (2 val)

- Stateless (sem estado entre pedidos)
- Cacheable
- Interface uniforme
- Sistema em camadas

- Dados trocados em texto (JSON ou XML), NÃO em binário

⚠ Armadilha (2021/22): REST permite XML, não só JSON. E NÃO usa formato binário.

JWT (JSON Web Token) ★★

Saiu em: Escolha múltipla + Aberta em TODOS os exames

```
const jwt = require('jsonwebtoken');

// Criar token (no login)
const token = jwt.sign({ userId: user._id, role: 'ADMIN' }, 'segredo', { expiresIn: '1h' });

// Verificar token (middleware)
jwt.verify(token, 'segredo', (err, decoded) => {
  // decoded contém os dados do utilizador
});
```

⚠ Armadilha: JWT NÃO obriga a usar REST API. Pode ser usado com qualquer tipo de aplicação.

FASE 4 — Angular & Full-Stack (≈3h de estudo)

🎯 Objetivo: Dominar Angular end-to-end — a parte mais "densa" do exame. 📄 PDFs: Client Side Frameworks.pdf, Angular Componentes e Serviços.pdf, Angular Components Forms Deploy.pdf, Angular autenticação e autorização.pdf, Deploy MEAN Stack Applications.pdf

4.1 — TypeScript & Angular Intro

TypeScript ★

- Superset de JavaScript — tudo que é JS é válido em TS
- Criado pela Microsoft
- Adiciona: tipos estáticos, compilação
- .ts → compilados para .js com tsc

⚠ Armadilha (2020/21 V/F): A linguagem do Angular é TypeScript, não JavaScript.

Angular CLI

```
npm install -g @angular/cli
ng new my-app # Cria projeto (NÃO é "npm new"!)
```

```
npm start # ou: ng serve --open
```

```
ng generate component nome # Cria componente (4 ficheiros)
```

```
ng generate service nome # Cria serviço
```

```
ng build # Gera build de produção em dist/
```

⚠ Armadilha (2024/25): npm new my-app NÃO existe! O correto é ng new my-app.

4.2 — Componentes Angular

Estrutura de um Componente (4 ficheiros)

- *.component.ts — lógica (TypeScript) → Controller (MVC)
- *.component.html — template → View (MVC)
- *.component.css — estilos
- *.component.spec.ts — testes

Data Binding ★★

Tipo	Sintaxe	Direção
Interpolation	{{ variavel }}	Component → View

Tipo	Sintaxe	Direção
Property binding	[propriedade]="valor"	Component → View
Event binding	(evento)="handler()"	View → Component
Two-way binding	[(ngModel)]="variavel"	Bidirecional ★

Ciclo de Vida do Componente ★

Hook	Quando
ngOnInit()	Após inicialização (1 vez) — o mais usado
ngOnChanges()	Quando inputs mudam
ngOnDestroy()	Antes de destruir (cleanup)

4.3 — Serviços Angular ★ ★ ★

Saiu em: Escolha múltipla 2024/25, 2021/22, 2020/21

O que são serviços:

- Classes TypeScript puras com @Injectable()
- NÃO têm template HTML (só componentes têm!)
- Injetados via Dependency Injection em múltiplos componentes
- Usam HttpClient para comunicar com APIs REST
- Usam Observables (RxJS) para programação reativa

```
@Injectable({ providedIn: 'root' })
export class RestService {
  constructor(private http: HttpClient) {}

  getProducts(): Observable<any> {
    return this.http.get('http://api/products');
  }
}
```

4.5 — Guards & Interceptors ★ ★ ★

Saiu em: Aberta 2024/25 (1.5 val), Escolha múltipla 2021/22

Guards (Guardas de Rota)

- Implementam CanActivate
- Controlam o acesso a rotas/páginas
- Se retornam true, navegação prossegue; false, bloqueia

Interceptors (Interceptadores HTTP)

- Implementam HttpInterceptor
- Modificam pedidos HTTP antes de serem enviados
- Uso principal: adicionar token JWT a TODOS os pedidos automaticamente

Guard vs Interceptor (DECORAR!)

	Guard	Interceptor
Protege	Rotas/páginas	Pedidos HTTP
Interface	CanActivate	HttpInterceptor
Quando age	Antes de navegar	Antes de enviar pedido HTTP
Exemplo	Bloquear acesso a /admin	Adicionar token JWT ao header

FASE 5 — Segurança & Revisão Final (≈2h de estudo)

🎯 **Objetivo:** Eliminar erros e consolidar tudo. Esta fase faz a diferença entre 16 e 20.

5.1 — Segurança em Aplicações Web

Validação de Dados ★★

Saiu em: V/F 2024/25, Aberta 2020/21 (3 val) — O tema MAIS valioso em respostas abertas!

A validação DEVE ser feita em AMBOS os lados — frontend E backend!

Onde	Porquê	Como
Frontend	Melhor UX (feedback imediato)	Atributos HTML (required), validação Angular
Backend	Segurança (o utilizador pode contornar o frontend!)	Middleware Express, Mongoose validators

Porquê validar no backend? (saber justificar):

- O utilizador pode desativar JavaScript no browser
- Pode enviar pedidos diretamente com Postman ou curl
- Pode manipular o HTML/JS no DevTools
- Confiar apenas no frontend = falha de segurança grave

Princípios de Segurança

- Nunca expor credenciais no código do frontend
- Nunca ligar o frontend diretamente a uma base de dados
- Sempre usar HTTPS em produção
- Sempre validar e sanitizar inputs no servidor
- Guardar passwords de forma hashed
- Usar JWT com segredo forte e tempo de expiração

5.2 — Identificar Tecnologia por Código ★★

Saiu em: Escolha múltipla em TODOS os exames

Vejo no código...	Tecnologia
@Injectable(), Observable, HttpClient	Angular Service
req, res, next, app.use()	Express Middleware
mongoose.Schema, mongoose.model	Mongoose (MongoDB)
document.getElementById(), addEventListener	JavaScript DOM
res.render('view', data)	Express + Template Engine
jwt.sign(), jwt.verify()	JWT (jsonwebtoken)
@NgModule, declarations, imports	Angular Module
<%= %>, <% %>	EJS (Template Engine)
@Component, ngOnInit	Angular Component
CanActivate	Angular Guard
HttpInterceptor, intercept	Angular Interceptor
BehaviorSubject, subscribe	RxJS / Angular Service

5.3 — Armadilhas de Escolha Múltipla (DECORAR!)

✗ Afirmações FALSAS frequentes

Afirmção falsa	Porquê é falsa
"HTML é uma linguagem de programação"	É de marcação
"CSS é uma linguagem de programação"	É de estilos
"Node.js é uma linguagem de programação"	É um runtime
"Angular é fullstack"	É client-side
"NPM cria componentes Angular"	Isso é ng generate
"NPM instala MongoDB"	Instala o driver (mongoose)
"Express executa no browser"	É server-side
"getElementById retorna todos os elementos"	Retorna UM
"Validação só no frontend é suficiente"	Validar sempre no backend
"Serviços Angular têm template HTML"	Não! Só componentes
"MongoDB pode correr no frontend"	BD corre no servidor
"JWT obriga a usar REST API"	Pode ser usado com qualquer aplicação
"REST só permite JSON"	Permite JSON e XML
"Login com GET"	Perigoso! Usar POST
"Angular executa no Node.js"	Executa no browser
"npm new my-app"	O correto é ng new my-app

✓ Afirmções VERDADEIRAS frequentes

Afirmção verdadeira	Justificação
"Angular permite reutilização de componentes"	Via @Input(), selectors, serviços
"Serviços Angular podem ser injetados em múltiplos componentes"	Via DI (@Injectable)
"Express suporta template engines"	EJS, Pug, Mustache (res.render())
"Mongoose permite aceder ao MongoDB"	É o driver/ODM Node.js para MongoDB
"npm start inicia um projeto"	Executa o script "start" do package.json
"Angular usa Observables"	Via RxJS (Observable, subscribe)
"Formulário HTML pode submeter sem JS"	<form> funciona nativamente
"JavaScript funciona no frontend e backend"	Browser + Node.js
"CSS formata o aspeto de páginas"	É a sua função exata

5.4 — Fluxo Completo MEAN (saber de cor)

- 1. Utilizador interage com Angular (Frontend/SPA no Browser)
- 2. Guard verifica se utilizador pode aceder à rota
- 3. Componente Angular usa Serviço com HttpClient
- 4. Interceptor adiciona token JWT ao header do pedido HTTP
- 5. Express recebe pedido → Middleware verifica JWT → Controller
- 6. Controller interage com MongoDB via Mongoose (Model)
- 7. Resposta JSON volta pelo Express → Angular
- 8. Angular atualiza a View via data binding (Observable → subscribe)

5.5 — O que cada ferramenta FAZ vs NÃO FAZ

Ferramenta	FAZ 	NÃO FAZ 
NPM	Instala packages, gere dependências, executa scripts (npm start)	NÃO cria componentes, NÃO instala MongoDB, NÃO compila TypeScript
Angular CLI (ng)	Cria projetos, componentes, serviços, faz build	NÃO gere packages (isso é o npm), NÃO corre no servidor
Express	Routing, middleware, serve páginas/API, usa template engines	NÃO corre no browser, NÃO é uma linguagem
Mongoose	Liga Node.js ao MongoDB, define schemas/models, CRUD	NÃO é uma base de dados (MongoDB é), NÃO instala MongoDB
JWT	Autenticação por tokens, assinatura digital, stateless	NÃO obriga a usar REST, NÃO guarda sessões no servidor

Checklist Final — Antes do Exame

- ☐ Sei explicar o modelo cliente-servidor (1-2 parágrafos)
- ☐ Sei os métodos HTTP e o seu mapeamento CRUD
- ☐ Sei distinguir 401 vs 403
- ☐ Sei que HTML e CSS NÃO são linguagens de programação
- ☐ Sei que Node.js é um runtime, não uma linguagem
- ☐ Sei a diferença var vs let vs const e == vs ===
- ☐ Sei que getElementById retorna UM elemento
- ☐ Sei usar localStorage para persistência (padrão getItem/setItem)
- ☐ Sei o que é middleware e que precisa de next()
- ☐ Sei as diferenças req.params vs req.query vs req.body
- ☐ Sei as 3 formas de adicionar CSS a uma página
- ☐ Sei o Box Model (content → padding → border → margin)
- ☐ Sei explicar o padrão MVC e aplicá-lo em Express
- ☐ Sei a diferença SQL vs NoSQL e o que é ACID
- ☐ Sei criar um schema/model Mongoose e fazer CRUD
- ☐ Sei o que é REST e as suas propriedades (stateless, etc.)
- ☐ Sei o que é CORS e como resolver
- ☐ Sei criar e verificar JWT (sign/verify)
- ☐ Sei que Angular é client-side e usa TypeScript
- ☐ Sei os 4 tipos de data binding Angular
- ☐ Sei o que são serviços Angular (NÃO têm template HTML!)
- ☐ Sei a diferença Guard vs Interceptor
- ☐ Sei explicar autenticação vs autorização com JWT
- ☐ Sei que a validação deve ser feita no frontend E backend
- ☐ Sei identificar tecnologia por excerto de código
- ☐ Sei o que é a MEAN Stack e o fluxo completo
- ☐ Sei corrigir 3 erros clássicos em HTML/JS (declaração, string, evento)
- ☐ Sei que NUNCA se liga o frontend diretamente à BD

 **Dica final:** Lê este documento 3 vezes — uma vez corrida, uma vez focando nas , e uma última vez apenas nas tabelas de armadilhas. Boa sorte! 